

AIM2- Preparations

Week 2, Day 3

Data Cleaning with Pandas

You've now built the foundation we need:

Week 1 → Python 

↓

Week 2 Day 1 → NumPy 

↓

Week 2 Day 2 → Pandas Fundamentals 

↓

Today → Data Cleaning 

Today we're going to work with something much closer to **real-world data**.

Real datasets are rarely perfect. You may encounter:

- Missing values
- Duplicate rows
- Wrong data types
- Extra spaces
- Inconsistent capitalization
- Invalid values

Before analyzing data or training an ML model, we usually need to **inspect and clean it**.

Today we'll focus **only on Data Cleaning**. We won't move into EDA or visualization yet.

Day 3 Goals

By the end of today, you should understand:

- What data cleaning means
- Missing values / NaN
- `isnull() / isna()`
- `notnull()`
- `dropna()`
- `fillna()`
- `duplicated()`
- `drop_duplicates()`
- `astype()`
- String cleaning
- `.str.strip()`
- `.str.lower()`
- `.str.title()`
- Replacing incorrect values
- Basic validation
- Why cleaning matters before ML

Then you'll clean a messy student dataset yourself.

1. What Is Data Cleaning?

Imagine someone gives you:

Name	Age	City	Score
Krishna	28	London	85
Rahul	NaN	Toronto	75
Priya	27	Ottawa	NaN
Krishna	28	London	85
Amit	32	london	70
Neha	29	" Toronto "	92

Immediately we have problems.

Missing data

Rahul → Age missing

Priya → Score missing

Duplicate

Krishna appears twice

Inconsistent text

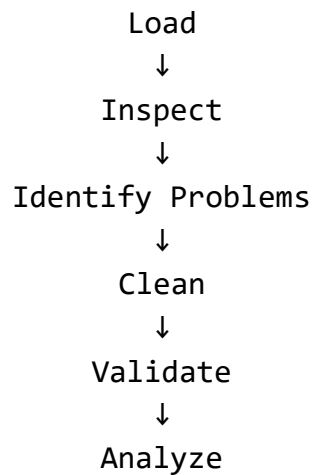
London
london

Extra whitespace

" Toronto "

If we start analyzing without fixing these problems, our results can be misleading.

A common workflow is:



2. Our Messy Dataset

Create today's file, perhaps:

day-03-data-cleaning.py

Start with:

```
import pandas as pd
import numpy as np
```

```
data = {
    "Name": ["Krishna", "Rahul", "Priya", "Krishna", "Amit", "Neha"],
    "Age": [28, np.nan, 27, 28, 32, 29],
    "City": ["London", "Toronto", "Ottawa", "London", "london", "Toronto "],
    "Score": [85, 75, np.nan, 85, 70, 92]
}

df = pd.DataFrame(data)

print(df)
```

You'll see NaN.

3. What Is NaN?

NaN means:

Not a Number

In data analysis, it commonly represents a **missing value**.

For example:

Name	Age
Krishna	28
Rahul	NaN

means Rahul's age is missing.

Don't think of NaN as automatically being 0.

These are completely different:

Score = 0

means we actually have a score of zero.

Whereas:

Score = NaN

means we don't have a score.

That distinction becomes very important in ML.

4. Detecting Missing Values — `isnull()`

Use:

```
print(df.isnull())
```

Pandas checks every cell.

You'll get Boolean values:

False → value exists

True → value is missing

This should remind you of yesterday's Boolean masks.

5. Count Missing Values

This is much more useful:

```
print(df.isnull().sum())
```

You might get something like:

```
Name      0
Age        1
City       0
Score      1
dtype: int64
```

Meaning:

Age → 1 missing value

Score → 1 missing value

Notice the chain:

```
df.isnull().sum()
```

First:

```
df.isnull()
```

creates the Boolean DataFrame.

Then:

```
.sum()
```

counts the True values in each column.

Why?

In Python numerical contexts:

True ≈ 1

False ≈ 0

So Pandas can count them.

6. `isna()` vs `isnull()`

You'll see both:

```
df.isna()
```

and:

```
df.isnull()
```

For normal Pandas usage, they do the same thing.

So:

```
df.isna().sum()
```

and:

```
df.isnull().sum()
```

are equivalent.

You'll commonly see `isna()` in data-cleaning code.

7. `notnull()`

The opposite is:

```
df.notnull()
```

It checks where values **exist**.

True → value exists

False → missing

Example:

```
print(df[df["Score"].notnull()])
```

This returns students who actually have a score.

Notice how yesterday's filtering knowledge appears again:

```
df[condition]
```

8. Removing Missing Values – dropna()

One possible strategy is simply to remove rows containing missing data.

```
clean_df = df.dropna()

print(clean_df)
```

Important:

```
df.dropna()
```

doesn't necessarily modify df itself.

We're storing the result:

```
clean_df = df.dropna()
```

 **But should we always delete missing data?**

No.

Imagine a dataset with:

1,000,000 rows

and only 5 unusable rows.

Removing them might be fine.

But imagine:

100 rows

40 rows have some missing information

Deleting all 40 could throw away a huge amount of useful information.

So:

Missing data needs a **decision**, not just a command.

9. Filling Missing Values – `fillna()`

Instead of deleting values, we can fill them.

Example:

```
df["Age"] = df["Age"].fillna(0)
```

But again—is 0 a sensible age?

Probably not.

A more reasonable strategy might be using the average age:

```
average_age = df["Age"].mean()  
  
df["Age"] = df["Age"].fillna(average_age)
```

- If Priya's real age is unknown and the mean is 29.4, putting 29 doesn't suddenly mean that Priya is actually 29. It means we estimated her missing age using the dataset mean.

If your business rule requires a whole-number age, you might intentionally round:

```
average_age = round(df["Age"].mean())
```

This means:

```
Calculate average of known ages  
    ↓  
Use it for missing age
```

You can do the same with Score:

```
average_score = df["Score"].mean()

df["Score"] = df["Score"].fillna(average_score)
```

This is called **imputation**.

For now, you only need the basic idea. We'll revisit preprocessing strategies when we reach machine learning.

10. Detecting Duplicates

Use:

```
print(df.duplicated())
```

You might get:

```
False
False
False
True
False
False
```

The True means Pandas considers that row a duplicate of an earlier row.

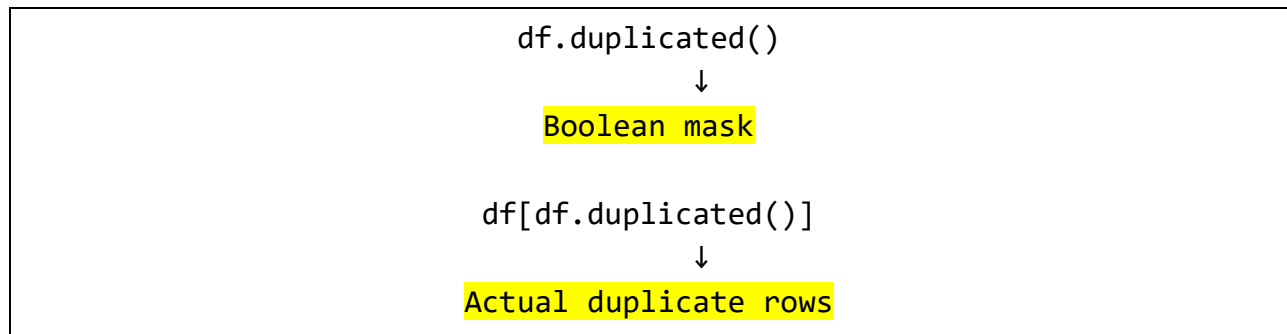
To count duplicates:

```
print(df.duplicated().sum())
```

And to actually view them:

```
print(df[df.duplicated()])
```

Notice the pattern again:



Exactly like yesterday.

11. Removing Duplicates

Use:

```
df = df.drop_duplicates()
```

Then:

```
print(df)
```

The duplicate row disappears.

You can verify:

```
print(df.duplicated().sum())
```

Hopefully:

0

12. Cleaning Extra Spaces

Consider:

" Toronto "

versus:

"Toronto"

To a human, they're obviously the same city.

To a computer, they're different strings.

Use:

```
df["City"] = df["City"].str.strip()
```

strip() removes spaces from the beginning and end.

Example:

" Toronto "

↓

"Toronto"

This should look familiar from your Week 1 string work.

Pandas uses:

.str

to apply string operations to a Series.

13. Fixing Capitalization

We currently have:

London

london

We can standardize them.

Lowercase

```
df["City"] = df["City"].str.lower()
```

Result:

london
toronto
ottawa

Title case

Or:

```
df["City"] = df["City"].str.title()
```

Result:

London
Toronto
Ottawa

For our dataset, title case is cleaner.

You can combine operations:

```
df["City"] = df["City"].str.strip().str.title()
```

Read that from left to right:

```
City
↓
remove spaces
↓
convert to title case
```

14. Cleaning Names

Same concept:

```
df["Name"] = df["Name"].str.strip().str.title()
```

So something messy like:

```
" krishna "
```

becomes:

```
"Krishna"
```

15. Replacing Incorrect Values

Imagine:

```
data = {  
    "City": ["London", "Toronto", "Unknown", "Ottawa"]  
}
```

Suppose "Unknown" should be considered missing.

You can do:

```
df["City"] = df["City"].replace("Unknown", np.nan)
```

Or imagine:

```
Londn
```

is clearly a typo for London:

```
df["City"] = df["City"].replace("Londn", "London")
```

This is useful for standardizing categories.

16. Wrong Data Types

This is another common real-world problem.

Imagine:

```
data = {  
    "Age": ["28", "30", "27", "32"]  
}
```

These look like numbers, but they're actually strings.

Check:

```
print(df.dtypes)
```

You may discover Age isn't numeric.

If the values are valid numbers:

```
df["Age"] = df["Age"].astype(int)
```

Now Age becomes an integer column.

You can also convert to:

```
.astype(float)  
.astype(str)
```

depending on what you need.

17. `astype()` Can Fail

Imagine:

```
data = {  
    "Age": ["28", "unknown", "27", "32"]  
}
```

Then:

```
df["Age"].astype(int)
```

will fail because:

"unknown"

cannot become an integer.

For messy real datasets, Pandas provides:

```
pd.to_numeric()
```

Example:

```
df["Age"] = pd.to_numeric(df["Age"], errors="coerce")
```

The invalid value:

"unknown"

becomes:

NaN

Then we can decide what to do with that missing value.

This gives us a useful cleaning workflow:

```
graph TD
    A["unknown"] --> B["pd.to_numeric(errors='coerce')"]
    B --> C["NaN"]
    C --> D["fillna() / dropna()"]
```

18. Basic Validation

Cleaning isn't finished just because the code ran.

After cleaning, inspect again:

```
print(df)
```



```
print(df.info())
print(df.isnull().sum())
print(df.duplicated().sum())
```

Actually, because you learned this yesterday, remember:

```
df.info()
```

rather than:

```
print(df.info())
```

because `info()` already prints its output.

Our goal after cleaning might be:

Missing values → 0

Duplicates → 0

Correct dtypes → 

Clean text → 

The Big Data-Cleaning Mental Model

Don't memorize random Pandas functions.

Think in terms of questions:

1. What does my data look like?

↓

`head()`, `info()`, `dtypes`

2. Is anything missing?

↓

`isna().sum()`

3. Are there duplicates?

↓

`duplicated().sum()`

4. Are the data types correct?



`dtypes`

5. Is text consistent?



`strip()`, `lower()`, `title()`

6. Are there invalid values?



`replace()`, `to_numeric()`

7. Clean the problems



`fillna()`, `dropna()`,
`drop_duplicates()`, etc.

8. Validate again

That's much closer to how a data analyst approaches a dataset.



Exercise 1 — Missing Values

Create:

```
data = {  
    "Name": ["Krishna", "Rahul", "Priya", "Amit", "Neha"],  
    "Age": [28, None, 27, 32, 29],  
    "Score": [85, 75, None, 70, 92]  
}
```

Create a DataFrame.

Then:

1. Print the DataFrame
2. Count missing values for every column

3. Print rows where Score is not missing
4. Calculate the mean Age
5. Fill the missing Age using the mean Age
6. Calculate the mean Score
7. Fill the missing Score using the mean Score
8. Print the cleaned DataFrame

Exercise 2 — Duplicates

Create:

```
data = {  
    "Name": ["Krishna", "Rahul", "Priya", "Krishna", "Amit"],  
    "Score": [85, 75, 91, 85, 70]  
}
```

Then:

1. Check duplicated()
2. Count duplicate rows
3. Display the duplicate rows
4. Remove duplicates
5. Verify the duplicate count is now 0

Exercise 3 — String Cleaning

Create:

```
data = {  
    "Name": [" krishna", "RAHUL ", "priya", " Amit "],  
    "City": [" london ", "TORONTO", "ottawa ", " LONDON"]  
}
```

Clean the data so the final result looks like:

Name	City
Krishna	London
Rahul	Toronto
Priya	Ottawa
Amit	London

Try using:

```
.str.strip()  
.str.title()
```



Exercise 4 — Incorrect Data Types

Create:

```
data = {  
    "Name": ["Krishna", "Rahul", "Priya", "Amit"],  
    "Age": ["28", "30", "27", "32"]  
}
```

Then:

1. Print dtypes
2. Convert Age to integer
3. Print dtypes again

Explain in a comment:

Why was Age not numeric originally even though it contained numbers?



Exercise 5 — Invalid Numeric Data

Create:

```
data = {
```

```
"Name": ["Krishna", "Rahul", "Priya", "Amit"],  
"Score": ["85", "unknown", "91", "70"]  
}
```

Try to think about why this would fail:

```
df["Score"].astype(int)
```

Then use:

```
pd.to_numeric(df["Score"], errors="coerce")
```

After conversion:

1. Print the DataFrame
2. Count missing values
3. Calculate the mean of valid scores
4. Fill the missing score with the mean
5. Print the cleaned DataFrame

Day 3 Mini-Project

Messy Student Dataset Cleaner

Here's your messy dataset:

```
import pandas as pd  
import numpy as np  
  
data = {  
    "Name": [  
        " krishna",  
        "RAHUL ",  
        "priya",  
        " krishna",  
        "Amit",  
        "NEHA "  
    ]  
}
```

```
],  
  
"Age": [  
    "28",  
    "30",  
    "unknown",  
    "28",  
    "32",  
    "29"  
],  
  
"City": [  
    " london ",  
    "TORONTO",  
    "ottawa",  
    " london ",  
    "LONDON",  
    " Toronto "  
],  
  
"Math": [  
    "85",  
    "75",  
    "91",  
    "85",  
    "65",  
    "88"  
],  
  
"Python": [  
    92,  
    80,  
    95,  
    92,  
    None,  
    85  
],  
  
"AI": [  

```

```
        88,  
        75,  
        94,  
        88,  
        68,  
        None  
    ]  
}  
  
df = pd.DataFrame(data)
```

This dataset intentionally contains several problems.

Your job

Clean it step by step:

1. Display the original dataset
2. Run `info()`
3. Display the original data types
4. Count missing values
5. Count duplicate rows
6. Clean Name
7. Clean City
8. Convert Age to numeric
9. Convert Math to numeric
10. Fill missing Age using its mean
11. Fill missing Python using its mean
12. Fill missing AI using its mean
13. Remove duplicate rows
14. Create an Average column from Math, Python and AI
15. Display the final cleaned DataFrame
16. Display final data types
17. Verify missing-value counts
18. Verify duplicate count

Suppose you have:

```
data = {  
    "Name": ["Krishna", "Rahul", "Priya", "Amit", "Neha"],  
    "City": ["London", "Toronto", None, "London", "unknown"]  
}
```

You **cannot calculate a mean** of cities:

```
df["City"].mean()    # ❌ Doesn't make sense
```

Instead, you choose a method based on what the missing value means.

1. Fill with the most common value — Mode

For categorical data, the equivalent idea to using the mean is often the **mode (most frequently occurring value)**.

```
city_mode = df["City"].mode()[0]  
print(city_mode)
```

If London appears most often:

London

then:

```
df["City"] = df["City"].fillna(city_mode)
```

Now the missing city becomes London.

But be careful: ***just because London is the most common city doesn't mean Priya actually lives in London. You're making an assumption.***

2. Keep it as "Unknown"

Sometimes this is actually the **better choice**.

```
df["City"] = df["City"].fillna("Unknown")
```

Result:

Krishna	London
Rahul	Toronto
Priya	Unknown
Amit	London
Neha	Unknown

This preserves the fact that **we don't know Priya's city** rather than inventing one.

This is especially useful when the missing category itself is meaningful.

3. What if the dataset already contains "unknown"?

Very important distinction:

None
np.nan

are recognized by Pandas as missing values.

But:

"unknown"
""
"N/A"
"missing"

may just be ordinary strings.

For example:

```
df["City"].isna()
```

will not necessarily consider:

"unknown"

missing.

So you may first standardize those values:

```
df["City"] = df["City"].replace(
```

```
[ "unknown", "Unknown", "" ],  
np.nan  
)
```

Then:

```
print(df["City"].isna().sum())
```

can detect them.

This is a **very common real-world data-cleaning step**.

4. Sometimes drop the row

If City is essential and only a tiny number of records are missing:

```
df = df.dropna(subset=["City"])
```

could be appropriate.

But again, don't blindly drop data. If you have 1,000,000 records and 10 are missing City, dropping them may be reasonable. If 40% of your dataset has no City, dropping all those rows could destroy a huge amount of useful information.

Numeric vs categorical — remember this

Problem	Possible strategy
Missing numeric value	Mean / Median / other appropriate value
Missing categorical value	Mode / "Unknown" / leave missing
Invalid numeric "unknown"	Convert to NaN, then decide
Invalid categorical "unknown"	Standardize to NaN or "Unknown"
Few unusable rows	Possibly dropna()
Many missing values	Investigate before dropping

So don't memorize:

NaN → fill with mean

Instead memorize:

NaN → understand what the column represents → choose an appropriate cleaning strategy.

For example, in your future housing project:

House Price missing

→ Maybe median/mean depending on the data

Bedrooms missing

→ Investigate; perhaps median/mode depending on meaning

City missing

→ "Unknown" may be safer

Province missing

→ Maybe infer it ONLY if another reliable field tells us

Property Type missing

→ Mode or "Unknown", depending on the analysis